

Contenido

1. Introducción al Desarrollo Web y HTML	2
1.1 ¿Qué es el Desarrollo Web?	2
Los tres pilares del desarrollo frontend.	2
El rol del desarrollador frontend:.....	2
1.2 Introducción a HTML (HyperText Markup Language).....	3
¿Qué es HTML?	3
Concepto de "marcado" y "etiquetas":.....	3
Breve historia:	4
2. La Estructura de un Documento HTML	5
2.1 Anatomía Básica de un Documento HTML	5
2.2 Elementos Esenciales dentro de <head>	6
3. HTML Semántico y Estructuración del Contenido	8
3.1 ¿Qué es el HTML Semántico y su Importancia?	8
Beneficios clave del HTML Semántico:.....	8
Evitar errores comunes:	8
3.2 Etiquetas Semánticas Comunes (para el <body>)	8
3.3 Contenedores Genéricos: <div> y	11
4. Elementos Interactivos y Multimedia.....	12
4.1 Enlaces (<a>).....	12
4.2 Imágenes ().....	12
4.3 Botones (<button>) y Entradas de Usuario (<input>, <textarea>).....	13
5. Introducción a CSS y Diseño Responsivo con Bootstrap	15
5.1 ¿Qué es CSS?	15
5.2 Fundamentos del Diseño Responsivo.....	16
5.3 Introducción a Bootstrap	17
5.4 El Sistema de Rejilla (Grid System) de Bootstrap.....	17
5.5 Clases Utilitarias de Bootstrap	19
5.6 Componentes y otras clases populares	20
6. Mejores Prácticas y Fundamentos de Accesibilidad.....	21
6.1 Errores Comunes en el Desarrollo Web (a evitar).....	21
6.2 Fundamentos de Accesibilidad Web (A11Y)	22

1. Introducción al Desarrollo Web y HTML

1.1 ¿Qué es el Desarrollo Web?

El **desarrollo web** es un proceso complejo y dinámico que implica la **creación de productos digitales en línea**. En esencia, se refiere a la construcción de **sitios web para Internet o una intranet**. Su propósito principal es dar vida a las experiencias digitales que encontramos y utilizamos a diario. En la actualidad, el negocio se mueve en la red, y el primer contacto que muchos usuarios tienen con un negocio, o incluso el único, es a través de una aplicación web o móvil. Por lo tanto, el desarrollo web no es solo una cuestión técnica, sino una estrategia crucial para la visibilidad y el éxito en el entorno digital.

El desarrollo web ha evolucionado significativamente desde los primeros días de la World Wide Web, cuando los sitios eran principalmente páginas HTML estáticas. Con el tiempo, creció la necesidad de sitios web más dinámicos e interactivos, lo que llevó al surgimiento de esta disciplina como un campo distinto.

Los tres pilares del desarrollo frontend.

El desarrollo web, particularmente el **desarrollo frontend**, se sustenta en tres lenguajes fundamentales que trabajan en conjunto para crear las interfaces que vemos e interactuamos:

1. HTML (HyperText Markup Language - Estructura):

Es el **esqueleto** de cualquier página web. Su función principal es **definir la estructura del contenido** y **proporcionar significado (semántica)**. Con HTML, indicamos qué parte del contenido es un párrafo, un título, una lista o una imagen. Aunque permite obtener un resultado visual, su rol principal no es el diseño o los colores. El HTML debe limitarse a definir la estructura del contenido, de modo que las etiquetas utilizadas nos permitan saber de un vistazo la importancia de cada parte.

2. CSS (Cascading Style Sheets - Presentación):

Se puede pensar en CSS como la **piel, el cabello o los ojos** del cuerpo humano, es decir, todo lo relacionado con la **estética visual**. Es el lenguaje que **describe la presentación del contenido HTML**, incluyendo el diseño, los colores, las fuentes y las variaciones en la interfaz para distintos dispositivos. Permite dar estilo a cualquier elemento HTML y, gracias a él, podemos crear diseños atractivos y responsivos.

3. JavaScript (Comportamiento/Interactividad):

Representa los **músculos** que proporcionan **movimiento y flexibilidad**. JavaScript es un lenguaje de programación que agrega **interactividad y funcionalidad dinámica** a las páginas web. Permite que los elementos de una página cambien o respondan a las acciones del usuario, como hacer clic en un botón, enviar un formulario o mostrar contenido adicional.

Estos tres lenguajes son los **fundamentales** para crear la interfaz de un sitio web o aplicación: HTML proporciona la estructura, CSS define la presentación y el estilo, y JavaScript agrega interactividad. La combinación de estos tres pilares es lo que permite construir experiencias digitales completas y atractivas. Es importante mantenerlos separados en archivos distintos (HTML por un lado, CSS por otro, y JavaScript al final del `<body>` para lograr un código limpio, organizado y fácil de mantener).

El rol del desarrollador frontend:

El **desarrollador frontend** juega un papel crucial en el equipo de desarrollo web. Su rol principal es **crear la interfaz de usuario (UI) y la experiencia de usuario (UX)** de un sitio web o aplicación. Esto significa que son los responsables de todo lo que el usuario ve e interactúa directamente en un sitio web.

Las responsabilidades de un desarrollador frontend incluyen:

- **Crear elementos visuales:** Diseñar y construir componentes como botones, menús, formularios y otros elementos interactivos.
- **Garantizar la funcionalidad:** Asegurarse de que estos elementos sean funcionales, es decir, que respondan como se espera a la interacción del usuario.
- **Asegurar la responsividad:** Optimizar la interfaz para que se adapte y funcione correctamente en **varios dispositivos** y tamaños de pantalla (ordenadores de escritorio, tabletas y móviles).
- **Mejorar la experiencia de usuario (UX):** Ir más allá de la simple estética y centrarse en que la **navegación sea intuitiva, fluida y anticipe las necesidades del usuario**. Una UX optimizada es clave para retener a los usuarios y convertirlos en clientes. El diseño de interfaz de usuario intuitivo y una experiencia de usuario optimizada garantizan que los visitantes puedan navegar fácilmente, mejorando las tasas de conversión.
- **Considerar la accesibilidad:** Implementar prácticas que permitan que las interfaces sean utilizables por **personas con discapacidades**, lo cual es fundamental para la equidad digital y una buena práctica ética y de SEO.

1.2 Introducción a HTML (HyperText Markup Language)

¿Qué es HTML?

HTML, o **Lenguaje de Marcado de Hipertextos**, es el lenguaje esencial y "MUY MUY sencillo de aprender" que se utiliza para **crear documentos en la web**. Su función principal es **describir la estructura del contenido** de una página web y **proporcionar significado (semántica)** a ese contenido.

Imagina una casa: HTML sería como los ladrillos y la estructura que le da forma. Con él, indicamos qué parte del texto es un párrafo, cuál es un título, dónde va una imagen o una tabla de datos. Es crucial entender que HTML se limita a definir la estructura y los elementos de una web, como el **esqueleto** del cuerpo humano, pero **no está diseñado para definir el diseño, colores o el aspecto visual** del mismo. Los desarrolladores deben evitar utilizar HTML de forma inadecuada para conseguir un resultado visual, ya que esto puede llevar a un "código terrible" aunque sea funcional. El código HTML debe limitarse a definir la estructura del contenido, de manera que, al ver las etiquetas, sepamos de inmediato la importancia de cada parte.

El término "Hipertexto" en HTML se refiere a los enlaces que conectan páginas web entre sí, ya sea dentro de un mismo sitio web o entre diferentes sitios. Estos enlaces son un aspecto fundamental de la Web, permitiendo la navegación y la interconexión de información.

Concepto de "marcado" y "etiquetas":

HTML utiliza el concepto de "marcado" para anotar texto, imágenes y otros contenidos que se mostrarán en un navegador web. Este marcado se realiza mediante **"elementos" especiales**.

Un elemento HTML se distingue del resto del texto en un documento a través de **"etiquetas"**. Una etiqueta consiste en el nombre del elemento rodeado por los caracteres < y >. La mayoría de los elementos HTML tienen una **etiqueta de apertura** y una **etiqueta de cierre**, que envuelven el contenido del elemento. La etiqueta de cierre se diferencia de la de apertura por incluir una barra diagonal (/) antes del nombre del elemento. Por ejemplo, para un título principal, usaríamos <h1> como etiqueta de apertura y </h1> como etiqueta de cierre.

Es importante notar que el nombre de un elemento dentro de una etiqueta **no distingue entre mayúsculas y minúsculas**, aunque por convención se suele escribir en minúsculas. Dentro de estas etiquetas, puede haber otras etiquetas, creando una estructura jerárquica para organizar el contenido.

Breve historia:

El lenguaje HTML fue **inventado por Tim Berners-Lee** en 1989, un físico del CERN. Su visión era crear un sistema de hipertexto para la web que facilitara la creación de documentos con referencias a otros contenidos y la colaboración entre equipos.

Desde entonces, HTML ha experimentado varias evoluciones. La última versión del estándar es **HTML5**, publicada en 2014. HTML5 no es solo una nueva versión del lenguaje, sino que representa un conjunto más amplio de tecnologías que permiten la creación de **sitios web y aplicaciones más diversas y potentes**. Esta evolución introdujo **nuevos elementos, atributos y comportamientos**, mejorando la semántica, el soporte multimedia, el rendimiento y la integración. Su importancia radica en que permite crear sitios web más rápidos, eficientes y accesibles. Aprender HTML5 y CSS3 es una **habilidad básica** y esencial para comenzar en el mundo del desarrollo web.

2. La Estructura de un Documento HTML

Todo documento HTML, por más sencillo que sea, debe seguir una estructura fundamental para que los navegadores web puedan interpretarlo y mostrarlo correctamente. Comprender esta anatomía básica es el primer paso para construir cualquier página web.

2.1 Anatomía Básica de un Documento HTML

La estructura básica de un documento HTML está compuesta por elementos clave que definen su tipo, su contenido raíz y las secciones para metadatos y contenido visible. Un ejemplo sencillo de un documento HTML nos muestra estos elementos esenciales:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Mi Página de Prueba</title>
  </head>
  <body>
    <!-- Contenido visible aquí -->
  </body>
</html>
```

A continuación, se detalla cada uno de estos elementos:

- **<!DOCTYPE html>: La declaración inicial que informa al navegador sobre el tipo de documento.**

Esta es la primera línea de cualquier documento HTML. Su función es esencialmente informar al navegador web qué tipo de documento está procesando. En HTML5, esta declaración es muy sencilla (<!DOCTYPE html>), a diferencia de versiones anteriores de HTML que requerían una declaración más compleja que referenciaba a un fichero DTD (Document Type Definition). Es una instrucción para el navegador sobre la versión del lenguaje de marcado utilizada.

- **<html>: El elemento raíz que envuelve todo el contenido de la página.**

Este es el elemento fundamental de un documento HTML. La etiqueta <html> es el **elemento raíz** que contiene absolutamente todo el contenido de la página web, desde los metadatos hasta el contenido visible. Es como el contenedor principal de todo el documento.

- **<head>: Contiene metadatos (información no visible para el usuario) como el título, los íconos, la codificación de caracteres. Solo puede haber un elemento <head> por documento.**

El elemento <head> actúa como un **contenedor para todo aquello que necesitamos incluir en nuestra página pero que no deseamos mostrar directamente a los usuarios**. Toda esta información es conocida como **metadatos** (información sobre la información). Aquí es donde se suelen definir aspectos como el título de la página, los íconos (favicons), el conjunto de caracteres utilizado, enlaces a hojas de estilo (CSS) y scripts (JavaScript). Es crucial recordar que **solo puede haber un elemento <head> por documento HTML**.

- **<body>: Contiene todo el contenido visible con el que los usuarios interactúan, como texto, imágenes, videos. Solo puede haber un elemento <body> por documento.**

El elemento <body> envuelve a **todo el contenido que se desea mostrar a los usuarios** cuando visitan la página. Esto incluye texto, imágenes, audio, video y cualquier otro elemento con el que los usuarios puedan interactuar. Al igual que con <head>, **solo puede haber un elemento <body> por documento HTML**. Es el "cuerpo" real de la página que el usuario experimenta.

2.2 Elementos Esenciales dentro de <head>

La sección <head> es un espacio técnico donde se configuran aspectos importantes de la página que afectan su comportamiento, su presentación y su interacción con motores de búsqueda y navegadores.

- **<title>: Define el título del documento, crucial para el SEO (posicionamiento en motores de búsqueda).**

Esta etiqueta es fundamental y **requerida en todos los documentos HTML**. El <title> define el título del documento, que es el texto que se muestra en la **barra de título del navegador o en la pestaña de la página**.

Su contenido es **muy importante para la optimización de motores de búsqueda (SEO)**. Los algoritmos de los motores de búsqueda utilizan el título de la página para decidir el orden en que se enumeran las páginas en los resultados de búsqueda. Un título claro y descriptivo mejora la visibilidad de la marca y el posicionamiento orgánico de un sitio web.

- **<meta>: Define metadatos como la descripción de la página, el autor, la codificación de caracteres (charset="utf-8") y la configuración del viewport para diseño responsivo.**

La etiqueta <meta> se utiliza para definir **información (metadatos) sobre el documento HTML**. Esta información no se muestra en la página, pero es utilizada por navegadores y motores de búsqueda.

Algunos de sus atributos más comunes y conocidos son:

- `description`: Define una breve descripción de la web, utilizada por los motores de búsqueda en los resultados.
- `charset="utf-8"`: Define la **codificación de los caracteres** del documento, asegurando que los caracteres especiales (como acentos o la 'ñ') se muestren correctamente.
- `name = "viewport" content = "width=device-width, initial-scale = 1.0"`: Esta es una configuración clave para el **diseño responsivo**. Indica al navegador que el ancho de la página debe adaptarse al ancho del dispositivo (`width=device-width`) y que la escala inicial debe ser 1.0 (`initial-scale=1.0`), lo que permite que el contenido se ajuste correctamente a diferentes tamaños de pantalla (escritorio, tabletas, móviles).

- **<link>: Se utiliza principalmente para enlazar hojas de estilo externas (por ejemplo, CSS).**

La etiqueta <link> define la relación entre el documento HTML actual y un recurso externo. Se usa comúnmente para **enlazar hojas de estilos externas (CSS)**, lo cual es una buena práctica para mantener el código organizado y separar la estructura del estilo.

El atributo `rel` (relationship) es obligatorio y define el tipo de relación (por ejemplo, `stylesheet` para hojas de estilo) y el atributo `href` indica la URL del recurso.

Por ejemplo: `<link href="main.css" rel="stylesheet">`.

- **<style>: Permite aplicar estilos CSS directamente dentro del HTML. Importante: Se recomienda evitar incrustar CSS directamente en el HTML en una web en producción, manteniendo los archivos separados.**

La etiqueta <style> se utiliza para **aplicar una hoja de estilos directamente en el documento HTML**. Permite definir reglas CSS que afectarán a los elementos de esa misma página.

Sin embargo, es una **práctica recomendada evitar incrustar CSS directamente dentro del HTML en una web en producción**. Esto se considera un "error típico" y "terrible", ya que mezcla el código CSS con el HTML, dificultando la legibilidad, el mantenimiento y la escalabilidad del proyecto. Lo ideal es mantener los archivos CSS por separado para un código limpio y organizado.

- **<script>: Permite integrar código JavaScript. Importante: Es preferible incluirlo al final del <body> o usar atributos defer o async para evitar bloquear el renderizado de la página.**

La etiqueta `<script>` permite **integrar código ejecutable del lado del cliente**, que normalmente es JavaScript. JavaScript es el lenguaje que añade interactividad y funcionalidad dinámica a las páginas web.

Aunque es posible añadir el código JavaScript directamente dentro de esta etiqueta (incrustado), es **preferible hacerlo a través de un fichero externo con extensión .js**.

Tradicionalmente, la etiqueta `<script>` suele situarse al final del `<body>` del documento HTML. ¿La razón? Para **impedir bloqueos al renderizar (parsear) el HTML**. Si los scripts se cargan en el `<head>` sin los atributos adecuados, el navegador debe descargar y ejecutar todo el JavaScript antes de continuar con el renderizado del HTML, lo que puede causar una mala experiencia de usuario al ralentizar la visualización de la página.

Existen alternativas para evitar este bloqueo al colocar los scripts en el `<head>`:

- **<script async>**: Permite descargar el script de forma asíncrona, pero **bloquea el análisis del HTML al ejecutarse**. Además, no garantiza que los scripts se ejecuten en el mismo orden en que están enlazados.
- **<script defer>**: El script se descarga de forma asíncrona, **en paralelo con el análisis del HTML**. Cuando el análisis del HTML finaliza, los scripts se ejecutan en el orden en que aparecen. Esta suele ser la opción más recomendada para enlaces a ficheros externos si el script no manipula el DOM antes de que la página se renderice.

3. HTML Semántico y Estructuración del Contenido

En esta sección, aprenderás a utilizar HTML de una manera más inteligente y efectiva, dando significado real al contenido de sus páginas web, lo cual es crucial en el desarrollo web moderno.

3.1 ¿Qué es el HTML Semántico y su Importancia?

El **HTML semántico** es el enfoque de escribir código HTML que **introduce significado o la semántica al contenido**, en lugar de solo definir cómo se ve (la presentación). Es decir, al usar HTML semántico, no solo estamos diciendo al navegador "muestra este texto", sino "este texto es un título principal", "esta es una sección de navegación", o "este es el contenido principal de un artículo".

Piensen en esto: Si construyen una casa, el HTML semántico es como usar los planos para indicar dónde está la sala, la cocina o los dormitorios. Cada parte tiene un propósito claro y una función específica.

Beneficios clave del HTML Semántico:

- **Mejora el SEO (Posicionamiento en Motores de Búsqueda):**

Los motores de búsqueda (como Google) rastrean e indexan las páginas web. Cuando el HTML es semántico, las etiquetas (como `<article>`, `<header>`, `<nav>`, `<footer>`) ayudan a los motores de búsqueda a **entender mejor el contenido y la estructura** de la página. Esto, a su vez, mejora el **posicionamiento orgánico** del sitio web en los resultados de búsqueda (SERPs). Un frontend bien estructurado con HTML semántico impacta positivamente en la visibilidad de la marca.

- **Mejora la Accesibilidad (A11Y):**

La accesibilidad se refiere a hacer que los sitios web sean **utilizables por el mayor número de personas posible**, incluyendo aquellas con discapacidades. Las tecnologías de asistencia, como los lectores de pantalla utilizados por personas con discapacidad visual, dependen del HTML semántico para interpretar el contenido de la página. Si una página está bien estructurada con etiquetas semánticas, un lector de pantalla puede anunciar "este es el título principal de la página" o "aquí comienza la navegación", facilitando enormemente la experiencia del usuario.

Evitar errores comunes:

Es un **error muy común y "terrible"** utilizar el lenguaje HTML de forma inadecuada, incluso si visualmente se obtiene un resultado similar. El código HTML debe limitarse a definir la estructura del contenido sin pararse a definir el aspecto visual del mismo.

- **Mal uso:** Usar una etiqueta `<p>` y aplicar estilos de fuente grandes (`<font size=24>`) para hacer que parezca un título. Esto es incorrecto porque un `<p>` significa "párrafo", no "título".

```
<p><font size=24>Nueva Thermomix</font></p>
```

- **Buen uso (semántico):** Utilizar la etiqueta `<h1>` para un título principal. Esto indica claramente que es el encabezado más importante de la página.

```
<h1>Nueva Thermomix</h1>
```

Al utilizar las etiquetas para el propósito para el que están diseñadas, los motores de búsqueda y las tecnologías de asistencia pueden entender la importancia de cada parte del contenido, lo que es la diferencia entre una página bien posicionada y otra mal posicionada, además de causar "fuertes dolores en la retina" a los desarrolladores experimentados que lo revisan.

3.2 Etiquetas Semánticas Comunes (para el `<body>`)

Las siguientes etiquetas son esenciales para estructurar el contenido de manera semántica dentro del `<body>` de un documento HTML:

➤ **<p>: Define párrafos de texto.**

Esta etiqueta se utiliza para mostrar bloques de texto. Es una de las etiquetas más básicas y semánticamente, indica un párrafo.

Ejemplo: `<p>Este es un párrafo de texto sobre HTML semántico. </p>`

➤ **<h1> a <h6>: Definen encabezados, estableciendo una jerarquía de importancia (H1 el más importante, H6 el menos). Crucial para estructurar el contenido.**

Estas etiquetas se utilizan para crear títulos y subtítulos. `<h1>` representa el título de mayor importancia (generalmente el título principal de la página), y `<h6>` el de menor importancia. Es fundamental usarlas jerárquicamente para estructurar el contenido lógicamente. Por lo general, los encabezados de mayor importancia tienen un tamaño de fuente mayor por defecto.

Ejemplo: `<h1>Título principal</h1> <h2>Subtítulo de sección</h2>`

➤ ** y : Para indicar importancia (negrita) y énfasis (cursiva) en el texto, respectivamente.**

`<strong>`: Se usa para definir una parte del texto que es **importante**. Tradicionalmente, los navegadores lo muestran en negrita, pero su significado es de importancia, no de estilo.

`<em>`: Se utiliza para **enfaticar** un texto. Normalmente se muestra en cursiva, pero al igual que `<strong>`, su función es semántica, no de estilo puro.

Ejemplo: `<p>Este es un texto con una parte <strong>importante </strong> y otra con <em>énfasis</em>.</p>`

➤ **<section>: Define secciones temáticas del documento.**

La etiqueta `<section>` se utiliza para agrupar contenido relacionado temáticamente. Cada `<section>` suele tener su propio encabezado (`h1-h6`) para indicar el tema de la sección.

Ejemplo:

```
<section>
  <h2>Sobre Nosotros</h2>
  <p>Aquí va la información de la empresa...</p>
</section>
```

➤ **<aside>: Contiene contenido indirectamente relacionado con el contenido principal.**

Esta etiqueta se usa para contenido que es "aparte" del contenido principal de la página o sección, pero que está indirectamente relacionado con él. Puede ser una barra lateral, citas al margen o información adicional.

Ejemplo:

```
<article>
  <p>El verano pasado fuimos a una playa increíble en Cádiz.</p>
  <aside>
    <h4>Playa de Cortadura</h4>
    <p>Un texto explicativo sobre la playa.</p>
  </aside>
  <p>Aparte de la playa, fuimos a cenar a unos cuantos sitios...</p>
</article>
```

➤ **<main>: Define el contenido principal y único del documento.**

La etiqueta <main> se usa para definir el **contenido principal o central** del documento. El contenido dentro de <main> debe ser **único** para ese documento y no debe contener elementos que se repiten en otras partes del sitio, como barras de navegación, menús laterales o pies de página. Solo puede haber una etiqueta <main> por documento.

Ejemplo: <main>Aquí iría el contenido principal y único de la página.</main>

➤ **<header> y <footer>: Representan la cabecera y el pie de una sección o del documento.**

<header>: Representa el **contenido introductorio** o un grupo de ayudas a la navegación para su ancestro más cercano (por ejemplo, el <header> de la página, un <section> o un <article>). Comúnmente contiene encabezados, logotipos, información de autoría y elementos de navegación.

<footer>: Representa el **pie de página** para su ancestro más cercano. Suele contener información de autoría, derechos de copyright, enlaces de contacto o enlaces a documentos relacionados.

Ejemplo:

```
<header><h1>Mi Blog Personal</h1></header>
```

```
<footer><p>&copy; 2025 Mi Blog</p></footer>
```

➤ **<article>: Para contenido independiente y auto-contenido, como una entrada de blog.**

Esta etiqueta se utiliza para secciones de contenido que son **independientes y auto-contenidas**. Esto significa que el contenido dentro de un <article> debería tener sentido por sí mismo y podría ser distribuido o reutilizado de forma independiente al resto de la página, como una entrada de blog, un comentario de usuario o un artículo de noticias.

Ejemplo:

```
<article>
  <header>
    <h2>Título del Artículo</h2>
  </header>
  <p>Contenido principal del artículo.</p>
</article>
```

➤ **<nav>: Contiene enlaces de navegación principales, como un menú.**

La etiqueta <nav> se utiliza para definir un bloque de **enlaces de navegación principales** en una página. No todos los enlaces de una página deben estar dentro de un <nav>, pero si hay un grupo de enlaces que forman una sección de navegación principal (como un menú de sitio, una tabla de contenidos o enlaces "siguiente/anterior"), deben estar contenidos en un <nav>.

Ejemplo:

```
<nav>
  <ul>
    <li><a href="/">Inicio</a></li>
    <li><a href="/servicios">Servicios</a></li>
    <li><a href="/contacto">Contacto</a></li>
  </ul>
</nav>
```

3.3 Contenedores Genéricos: <div> y

Además de las etiquetas semánticas, HTML proporciona elementos genéricos que sirven como contenedores cuando no hay una etiqueta semántica más adecuada para el propósito. Sin embargo, su uso debe ser consciente.

- **<div>: Un contenedor de bloque genérico. No es semántico y no se recomienda abusar de él; sustituir por etiquetas semánticas cuando sea posible.**

La etiqueta <div> (de "division") se utiliza como un **contenedor de bloque genérico** para agrupar otros elementos HTML. Es muy versátil y puede contener texto, otras etiquetas o cualquier tipo de contenido.

No es semántica. Esto significa que un <div> no aporta ningún significado intrínseco sobre el tipo de contenido que contiene. Para un navegador o un lector de pantalla, un <div> es simplemente un bloque sin propósito específico.

No se recomienda abusar de él. Es crucial **sustituir el <div> por etiquetas semánticas** siempre que sea posible para mejorar la legibilidad del código, el SEO y la accesibilidad. Por ejemplo, en lugar de <div> para una cabecera, se debería usar <header>.

Un elemento <div> siempre empieza en una nueva línea y ocupa todo el ancho disponible por defecto.

Ejemplo de uso:

```
<div>
  <h2>Bienvenida</h2>
  <p>Hola a todos.</p>
</div>
```

- **: Un contenedor inline utilizado para envolver parte de un texto y aplicarle estilos o atributos específicos.**

La etiqueta es un **contenedor inline genérico** que se utiliza para envolver una pequeña parte de texto o un documento. A diferencia de <div>, no inicia una nueva línea y solo ocupa el ancho necesario para su contenido.

Se usa principalmente para aplicar estilos específicos (a través de una clase o ID) o atributos a un fragmento de texto sin afectar el flujo del documento.

No es semántico. Al igual que <div>, no transmite significado sobre su contenido.

Ejemplo de uso:

```
<p>En esa empresa hay <span class="workers-number">25</span>
trabajadores.</p>
```

Donde la clase `workers-number` podría darle un color o tamaño específico solo al número.

- **Comprender la diferencia entre elementos de bloque (div, p, h1) e inline (span, strong, em).**

- **Elementos de Bloque (block):**

- Siempre comienzan en una nueva línea.
- Toman todo el ancho disponible del contenedor padre por defecto.
- Ejemplos incluyen <div>, <p>, <h1> a <h6>, <section>, <article>, <header>, <footer>, <nav>, <main>, <aside>.

- **Elementos Inline (inline):**

- No comienzan en una nueva línea.
- Solo ocupan el ancho necesario para su contenido.
- Ejemplos incluyen , , , <a>, , <button>, <input>.

4. Elementos Interactivos y Multimedia

En esta sección, aprenderás a incorporar elementos que permiten a los usuarios navegar por el sitio, interactuar con el contenido e incluso proporcionar información, así como a integrar imágenes para enriquecer la experiencia visual.

4.1 Enlaces (<a>)

La etiqueta <a> es fundamental en HTML, ya que define un **hipervínculo** o "anchor" (ancla), que es lo que permite **conectar páginas web entre sí**. El concepto de "Hipertexto" en HTML se refiere precisamente a estos enlaces que conectan documentos web, ya sea dentro de un mismo sitio o entre distintos sitios en la World Wide Web. La capacidad de vincular contenido hace que los usuarios se conviertan en participantes activos de la web.

➤ **Atributo href: La dirección del enlace.**

El atributo más importante de la etiqueta <a> es href (del inglés "Hypertext Reference"), el cual especifica la **dirección (URL)** a la que apunta el enlace. Por ejemplo, href="main.css" indica la ruta a una hoja de estilos.

➤ **Tipos de enlaces:** La versatilidad de la etiqueta <a> permite crear diversos tipos de enlaces para diferentes propósitos:

- **A otras páginas o sitios web:** Es el uso más común, donde el href contiene la URL completa de otra página en internet.

Ejemplo: Visita nuestros ejemplos.

- **A secciones específicas dentro de la misma página (#id):** Los enlaces pueden dirigir a los usuarios a una parte específica de la misma página. Esto se logra utilizando un **fragmento de URL** (una cadena de caracteres que refiere a un recurso subordinado o incluido) que comienza con el carácter #, seguido del id del elemento al que se desea enlazar. El elemento de destino debe tener un atributo id que coincida con el valor después del # en el href.

Ejemplo:

Un enlace Volver arriba dirige al usuario a un elemento con id="top" en el mismo documento.

- **A direcciones de correo electrónico (mailto:) o números de teléfono (tel:):** La etiqueta <a> también puede iniciar acciones fuera de la navegación web, como redactar un correo electrónico o hacer una llamada telefónica, utilizando esquemas de URL especiales.

- Para correos electrónicos: Se usa el esquema mailto: seguido de la dirección de correo.

Ejemplo: Contáctenos!.

- Para números de teléfono: Se usa el esquema tel: seguido del número de teléfono.

Ejemplo: Llámenos!.

4.2 Imágenes ()

Las imágenes son un componente visual fundamental en las páginas web. La etiqueta es un "elemento void", lo que significa que solo tiene etiqueta de apertura y **no debe tener etiqueta de cierre** ya que no contiene contenido.

➤ **Insertar imágenes: Atributo `src` (fuente de la imagen).**

Para insertar una imagen, se utiliza la etiqueta `<img>` junto con el atributo `src` (source), que especifica la **ruta o URL de la imagen** que se desea mostrar.

Ejemplo: ``. La ruta puede ser local o una URL de Internet.

➤ **Atributo `alt`: Texto alternativo descriptivo. Fundamental para la accesibilidad (lectores de pantalla) y el SEO.**

El atributo `alt` (alternative text) es **crucial y obligatorio** para cualquier imagen. Proporciona un **texto alternativo descriptivo** que se muestra si la imagen no se puede cargar (por problemas de red o si la ruta es incorrecta).

Su importancia principal radica en la **accesibilidad**: los lectores de pantalla (utilizados por personas con discapacidad visual) leen este texto para describir la imagen al usuario, garantizando que el contenido sea comprensible para todos.

Además, el atributo `alt` es **fundamental para el SEO**, ya que los motores de búsqueda utilizan este texto para comprender el contenido de la imagen y mejorar el posicionamiento de la. Las pautas WCAG (Web Content Accessibility Guidelines) exigen que todo contenido no textual tenga una alternativa textual.

➤ **Optimización de imágenes: Usar formatos adecuados (como WebP) y comprimirlas para mejorar la velocidad de carga de la página. Evitar usar fotos "pesadas" tal cual de la cámara.**

La **optimización de imágenes** es una de las "mejores prácticas" en el desarrollo web porque las imágenes suelen ser los **elementos más pesados** de una página, lo que puede ralentizar significativamente el tiempo de carga.

Para mejorar la velocidad, se recomienda:

- Utilizar **formatos de imagen de próxima generación** como **WebP**, que ofrecen una alta calidad con un menor tamaño de archivo en comparación con JPG o PNG.
- **Comprimir** las imágenes y ajustar sus dimensiones correctamente para reducir su peso sin sacrificar la calidad visual.
- Implementar la **carga diferida (lazy loading)** para imágenes que no están en la vista inicial de la página (fuera de la pantalla del usuario), lo que permite que el navegador las cargue solo cuando el usuario las necesita, mejorando la velocidad inicial del sitio. Evitar el uso de fotos "pesadas" directamente de la cámara.

➤ **`class="img-fluid"` (con Bootstrap): Para hacer que las imágenes sean responsivas y se adapten al tamaño del contenedor.**

En el contexto del diseño responsivo, especialmente al usar un framework como **Bootstrap**, la clase `img-fluid` es esencial. Al aplicarla a una imagen, esta se ajustará automáticamente al ancho de su contenedor, **escalando fluidamente** para adaptarse a diferentes tamaños de pantalla (móviles, tabletas, escritorios) sin desbordarse. Esto es parte del diseño responsivo inmediato que ofrece Bootstrap.

4.3 Botones (`<button>`) y Entradas de Usuario (`<input>`, `<textarea>`)

Estos elementos permiten a los usuarios interactuar con la página, enviar información y desencadenar acciones.

➤ **`<button>`: Creación de botones interactivos.**

La etiqueta `<button>` se utiliza para crear un **botón interactivo** en la interfaz de usuario. Estos botones pueden ser configurados para responder a eventos, como un `click`, mediante JavaScript.

Ejemplo:

```
<button onclick="handleOnClick()">Click me</button>.
```

➤ **<input>: Campos de entrada para que el usuario ingrese información.**

La etiqueta `<input>` se utiliza para crear **campos donde el usuario puede introducir información**. Es un elemento "void" o "vacío", lo que significa que **no requiere una etiqueta de cierre**.

➤ **Tipos: text, number, email, password, checkbox, radio, submit, etc.**

El atributo `type` es fundamental para definir la naturaleza del campo de entrada. Puede ser `text` para texto simple, `number` para números, `email` para direcciones de correo electrónico, `password` para contraseñas (que oculta los caracteres), `checkbox` para selección múltiple, `radio` para selección única, `submit` para enviar formularios, entre otros.

Ejemplo

```
type="number": <input type="number" placeholder="Introduzca un número">.
```

➤ **placeholder: Texto que se muestra cuando el campo está vacío.**

El atributo `placeholder` proporciona un **texto sugerido o un ejemplo de formato** que se muestra dentro del campo de entrada cuando este está vacío. Desaparece automáticamente cuando el usuario comienza a escribir.

Ejemplo:

```
<input type="number" placeholder="Introduzca un número">.
```

➤ **<textarea>: Área de texto multilínea para comentarios o mensajes largos.**

La etiqueta `<textarea>` se utiliza para crear un **campo de entrada de texto multilínea**, ideal para que los usuarios escriban comentarios largos, mensajes o descripciones. Se diferencia de `<input type="text">` en que permite múltiples líneas de texto y el usuario puede redimensionarlo por defecto.

○ **Solución al problema de redimensionamiento con CSS: `resize: none`.**

Un problema común con los `<textarea>` es que los usuarios pueden **redimensionarlos libremente**, lo que podría "arruinar toda la presentación" de la página.

Para evitar esto, se puede aplicar la propiedad CSS `resize: none;` al `textarea`. Esto deshabilita la capacidad del usuario para cambiar el tamaño del área de texto.

➤ **<label>: Etiquetas asociadas a los campos de entrada para mejorar la accesibilidad.**

La etiqueta `<label>` se utiliza para asociar una **descripción textual** a un control de formulario (como `<input>`, `<textarea>`, `<select>`). Esta asociación se realiza mediante el atributo `for` en la etiqueta `<label>`, cuyo valor debe coincidir con el atributo `id` del control de formulario.

El uso de `<label>` es una **práctica fundamental para la accesibilidad**. Ayuda a los usuarios de lectores de pantalla a entender el propósito de cada campo, ya que el lector puede leer la etiqueta asociada al campo. Además, al hacer clic en el texto del `<label>`, el foco se mueve automáticamente al campo de entrada asociado, mejorando la usabilidad para todos los usuarios.

Las WCAG 2.2 enfatizan que "se proporcionan etiquetas o instrucciones cuando el contenido requiere la intervención del usuario".

5. Introducción a CSS y Diseño Responsivo con Bootstrap

Esta sección te introducirá a las herramientas fundamentales para darle estilo y adaptabilidad a tus páginas web, dos aspectos cruciales en el desarrollo frontend moderno.

5.1 ¿Qué es CSS?

- **Lenguaje para la presentación y diseño visual del contenido HTML.**

CSS (Cascading Style Sheets) es un lenguaje de diseño gráfico que permite **definir la apariencia visual de los elementos HTML** que componen las interfaces web. Su propósito principal es describir cómo se deben representar los elementos HTML, tanto visualmente (diseño, colores, fuentes, animaciones) como de forma auditiva (para personas con discapacidad visual que usan tecnologías de asistencia). Si HTML proporciona la estructura del contenido, CSS se encarga de la capa visual. Se puede entender que, si el HTML es el esqueleto de un cuerpo humano, el CSS sería la piel, el cabello y los ojos, es decir, la estética visual.

- **La importancia de separar el HTML (estructura) del CSS (estilo) para un código limpio y organizado.**

Es crucial que el **código HTML se limite a definir la estructura y el significado semántico del contenido**, sin detenerse en su aspecto visual. Mezclar el código CSS con el código HTML es un error muy común y **terrible** para la organización y limpieza del código. Aunque los navegadores pueden procesar códigos mezclados, una práctica correcta en desarrollo web en producción es tener **ficheros HTML por un lado y ficheros CSS por otro, totalmente separados**. Esta separación no solo mejora la **legibilidad y el mantenimiento del código**, sino que también influye directamente en el **posicionamiento de la página en motores de búsqueda**. Además, el código resultante siempre será mucho más limpio y rápido.

- **Conceptos básicos de reglas CSS: selectores, propiedades y valores.**

El código CSS se compone de **reglas**. Una regla es un conjunto de propiedades que se aplicarán a un elemento determinado. En cada regla CSS distinguimos el **selector** y la **declaración**.

El **selector** se utiliza para referenciar los elementos a los que se desean aplicar estilos. Existen distintos tipos de selectores, como:

- **Selector de tipo (o etiqueta):** Aplica estilos a todos los elementos de un tipo específico (ej. `div { ... }`).
- **Selector de clase:** Aplica estilos a todos los elementos que tienen una clase específica (ej. `.example { ... }`).
- **Selector de ID:** Aplica estilos al elemento con un ID único. Solo se selecciona un elemento porque el ID debe ser único en un documento HTML (ej. `#example { ... }`).
- **Selector universal:** Selecciona todos los elementos y puede combinarse para selecciones más complejas.
- También existen combinadores para selecciones más complejas, como los **combinadores de hermanos, hijo y descendientes**.
- La **declaración** engloba un listado de **pares propiedad-valor**, encerrado entre llaves `{ }` y separado por punto y coma `;`.
 - Una **propiedad** es el atributo de estilo que se desea modificar (ej. `color`).
 - Un **valor** es el ajuste específico para esa propiedad (ej. `red`). Por ejemplo, `h1 { color: red; font-size: 24px; }`.
- **Unidades de longitud: Introducción a unidades relativas (em, rem, vw, vh, %) que escalan mejor en pantallas diferentes, a diferencia de las unidades absolutas como px.**

En CSS, existen diversas unidades para expresar una longitud.

- Las **unidades absolutas** (como `px` para píxeles, `cm` para centímetros, `mm` para milímetros, `in` para pulgadas, `pt` para puntos y `pc` para picas) no se recomiendan para un diseño responsivo, ya que **no se adaptan de igual manera a todas las pantallas**.
- Las **unidades relativas** son las más utilizadas en el diseño web flexible, ya que expresan una longitud relativa a otra propiedad o elemento, y **escalan mejor al renderizar en distintos tipos de pantalla**. Las más comunes son:
 - **em**: Es relativa a la propiedad `font-size` del elemento padre. Por ejemplo, `2em` significa dos veces el tamaño de la fuente del padre.
 - **rem**: Es relativa al `font-size` del elemento raíz (`<html>`). Si no se define, toma un valor por defecto de 16 píxeles. Por ejemplo, `0.8rem` resultaría en `12.8px` si el tamaño base es `16px`.
 - **vw**: Relativa al 1% del ancho del `viewport` (el tamaño de la ventana del navegador).
 - **vh**: Relativa al 1% del alto del `viewport`.
 - **%**: Relativa al elemento padre.

5.2 Fundamentos del Diseño Responsivo

- **¿Qué es el diseño responsivo?: Un enfoque que permite que las páginas web se adapten a diferentes tamaños de pantalla y dispositivos (ordenadores, tablets, celulares).**

El **diseño responsivo (Responsive Design)** es un enfoque que busca desarrollar y diseñar sitios web que **puedan ajustarse a cualquier resolución**, adaptando el layout y las imágenes a diferentes dispositivos para que el usuario tenga una experiencia satisfactoria. Esto significa que el contenido de una página web **se adapta a la pantalla de escritorio, a una tablet o a un celular (smartphone o móvil)**. Las aplicaciones web progresivas (PWA) son un ejemplo de esta adaptabilidad, ofreciendo una experiencia fluida e independiente del dispositivo o conexión.

- **Importancia: Mejora la experiencia del usuario y el posicionamiento en motores de búsqueda (Google favorece sitios responsivos).**
 - La importancia del diseño responsivo es multifacética:
 - **Mejora la experiencia del usuario:** Un sitio responsivo asegura que el contenido sea **accesible y utilizable para todas las personas** en cualquier dispositivo. Los usuarios móviles, por ejemplo, tienen una tasa de rebote del 61% si la experiencia es mala.
 - **Posicionamiento en motores de búsqueda (SEO): Google favorece el posicionamiento de los sitios con Responsive Design.** De hecho, una actualización del algoritmo de Google que priorizaba la adaptabilidad móvil fue apodada "Mobilegeddon". Un frontend bien estructurado con prácticas de SEO, como HTML semántico, mejora el posicionamiento orgánico.
 - También contribuye a **aumentar la velocidad de carga** y mejora la difusión en redes sociales. Es un elemento crítico en el desarrollo frontend y no solo una opción ética, sino una necesidad.
- **Introducción a Media Queries: Reglas CSS que aplican estilos específicos según las características del dispositivo (ancho, altura, orientación).**
 - Las **Media Queries** son reglas CSS que aplican estilos específicos **según las características del dispositivo** que se utilice para acceder a la web, como el ancho, la altura o la orientación de la pantalla. Permiten presentar **distintos layouts** o diseños dependiendo del tamaño o tipo de pantalla.
 - Bootstrap, por ejemplo, maneja **puntos de quiebre (breakpoints)** que son tamaños de pantalla predefinidos en píxeles. Estos puntos le indican a Bootstrap cómo distribuir los elementos para que sean responsivos en pantallas extra pequeñas, pequeñas, medianas, grandes, extra grandes y extra extra grandes.
 - Por ejemplo, se puede establecer que cuando el ancho de la pantalla sea menor de 600 píxeles, el fondo de la página cambie a azul claro, o el tamaño de la fuente se modifique.

5.3 Introducción a Bootstrap

- **¿Qué es Bootstrap? Un framework front-end que facilita la creación de sitios web atractivos y responsivos sin escribir CSS desde cero.**

Bootstrap es un **framework front-end** de código abierto y gratuito que facilita enormemente la creación de **sitios web atractivos y responsivos sin la necesidad de escribir CSS desde cero**. Es una **librería de CSS** que simplifica tareas de disposición y ubicación de contenidos, haciéndolas más fáciles. Proporciona un conjunto de herramientas que incluyen un **sistema de rejilla flexible**, **clases predefinidas** para estilos y estructuras, y **componentes reutilizables** como botones, tarjetas, modales y formularios. Su facilidad de uso y compatibilidad con múltiples navegadores lo convierten en una opción popular.

- **Ventajas: Ahorro de tiempo, diseño responsivo inmediato, gran comunidad y documentación.**
 - Las principales ventajas de utilizar Bootstrap son:
 - **Ahorro de tiempo:** Permite desarrollar interfaces modernas sin necesidad de diseñarlas desde cero. Puedes estructurar una página en pocas horas aplicando estilos modernos.
 - **Diseño responsivo inmediato:** Los componentes de Bootstrap se adaptan automáticamente a cualquier dispositivo, lo que lo convierte en un aliado clave para proyectos donde el tiempo es crucial.
 - **Amplia comunidad y documentación:** Cuenta con una vasta comunidad de desarrolladores y una documentación detallada y activa, lo que facilita el aprendizaje y la resolución de problemas.
 - **Integración:** Es compatible con otros frameworks y CMS como Angular, React o WordPress.
- **Cómo incluir Bootstrap en el proyecto: Usando una CDN (Content Delivery Network), que es la opción más rápida y recomendada, o descargando los archivos.**
 - Para incluir Bootstrap en un proyecto, tienes dos opciones principales:
 - **CDN (Content Delivery Network - Red de Distribución de Contenidos):** Esta es la **opción más rápida y recomendada**. Una CDN permite compartir recursos (como los documentos CSS y JavaScript de Bootstrap) a través de internet mediante una simple URL. Se importan los enlaces CSS y JavaScript directamente en tu archivo HTML.
 - **Descargando los archivos:** Puedes descargar los archivos de Bootstrap y alojarlos directamente en tu servidor.
 - Una vez incluido, Bootstrap se basa en el **uso de clases CSS predefinidas** para aplicar estilos y comportamientos a los elementos HTML.
- **Bootstrap 5: La versión más reciente, que eliminó la dependencia de jQuery y mejoró el CSS Grid.**
 - **Bootstrap 5** es una versión reciente del framework que ha evolucionado significativamente. Entre sus características más destacadas, **eliminó la dependencia de jQuery**, lo que lo hizo más ligero y rápido. También introdujo **mejoras en su sistema CSS Grid**, ofreciendo mayor control sobre el diseño de columnas y contenedores, y nuevos componentes más accesibles. Aunque el video de YouTube menciona la versión 5.2.2 en su fecha de grabación, el blog oficial de Bootstrap muestra actualizaciones más recientes, como la versión 5.3.8 en agosto de 2025. La comunidad también anticipa una futura **Bootstrap 6** con más optimización y compatibilidad con nuevas tecnologías.

5.4 El Sistema de Rejilla (Grid System) de Bootstrap

El sistema de rejilla de Bootstrap es una de sus características más poderosas, permitiéndote crear diseños responsivos de manera sencilla al organizar el contenido en una estructura de columnas y filas que se adapta a diferentes tamaños de pantalla.

- **Containers: Contenedores para centrar el contenido y darle un ancho adaptable (`.container` para ancho fijo en *breakpoints*, `.container-fluid` para ancho completo).**

- Los **contenedores (containers)** son bloques fundamentales que estructuran y centran el contenido de una página web en Bootstrap. Proporcionan un espaciado automático tanto a la izquierda como a la derecha, y alineaciones para diferentes dispositivos o *viewports*.
- La clase `.container` es el contenedor por defecto y se comporta de manera predeterminada para cada tipo de pantalla. Por ejemplo, en pantallas extra pequeñas, ocupa el 100% del ancho. Para pantallas pequeñas, tiene un ancho fijo de 540 píxeles, y así sucesivamente para tamaños medianos, grandes y extra grandes, aplicando márgenes laterales para centrar el contenido. Esto permite que el contenido esté de alguna manera centrado y con espacios a los lados.
- En contraste, la clase `.container-fluid` hace que el contenido ocupe **todo el ancho de la pantalla** disponible, desde el borde izquierdo hasta el derecho. Un cambio simple entre `.container` y `.container-fluid` puede modificar toda la estructura horizontal de tu página web.

➤ **Rows (`.row`): Filas que actúan como contenedores para las columnas.**

Las **filas (`.row`)** son elementos cruciales en el sistema de rejilla de Bootstrap. Su función principal es actuar como **contenedores para las columnas**. Dentro de una fila, puedes organizar uno o más elementos de columna, que se distribuirán horizontalmente. Un ejemplo práctico muestra cómo usar un `div` con la clase `row` para organizar contenido.

➤ **Columns (`.col`, `.col-sm`, `.col-md`, `.col-lg`, `.col-xl`): El sistema de rejilla de Bootstrap divide la pantalla horizontalmente en 12 partes. Las clases de columna permiten especificar cuántas de esas partes ocupa un elemento en diferentes tamaños de pantalla.**

- El sistema de rejilla de Bootstrap se basa en una división horizontal de la pantalla en **12 partes o unidades iguales**. Las clases de columna (`.col`, `.col-sm`, etc.) te permiten especificar **cuántas de estas 12 partes ocupa un elemento** en particular.
- Por ejemplo, si tienes tres columnas dentro de una fila y no especificas el tamaño, cada columna ocupará automáticamente 4 de las 12 partes ($4+4+4 = 12$), distribuyéndose equitativamente.
- Bootstrap utiliza **clases con prefijos de tamaño de pantalla** para que puedas controlar el ancho de las columnas en diferentes *breakpoints*:
 - `.col` (para todos los tamaños, *extra small* por defecto).
 - `.col-sm` (para pantallas pequeñas, *small*).
 - `.col-md` (para pantallas medianas, *medium*).
 - `.col-lg` (para pantallas grandes, *large*).
 - `.col-xl` (para pantallas extra grandes, *extra large*).
 - `.col-xxl` (para pantallas extra extra grandes).
- Puedes especificar que para pantallas pequeñas (`sm`), un elemento ocupe 8 espacios (`.col-sm-8`) y otro ocupe los 4 restantes (`.col-sm-4`). Este sistema proporciona una flexibilidad increíble para el diseño responsivo.

➤ **Breakpoints: Puntos de quiebre predefinidos por Bootstrap para extra pequeñas (celular), pequeñas (tablet), medianas, grandes y extra grandes (escritorio).**

- Los **breakpoints (puntos de quiebre)** son tamaños de pantalla predefinidos que Bootstrap utiliza para determinar cómo se deben distribuir y mostrar los elementos. Estos puntos le indican a Bootstrap la disposición de los elementos para asegurar la responsividad en diferentes dispositivos. Se basan en la unidad de medida de píxeles.
- Los puntos de quiebre predefinidos son:
 - **Extra pequeñas (celular/smartphone):** Se basa en un máximo de 576 píxeles de ancho.
 - **Pequeñas (tablet):** A partir de 576 píxeles de ancho.
 - **Medianas:** A partir de 768 píxeles de ancho.
 - **Grandes:** A partir de 992 píxeles de ancho.
 - **Extra grandes (escritorio):** A partir de 1200 píxeles de ancho.
 - **Extra extra grandes:** A partir de 1400 píxeles de ancho.

- Aunque estas configuraciones se pueden personalizar usando un precompilador de CSS como Sass, se recomienda mantener los valores por defecto, ya que son casi un estándar en la industria.
- **Ejemplo práctico: Crear un layout que muestre 3 columnas en escritorio, 2 en tablet y 1 en celular.**
 - Para crear un *layout* que se adapte a diferentes tamaños de pantalla, mostrando 3 columnas en escritorio, 2 en tablet y 1 en celular, se utilizan las clases de columna responsivas.
 - Por ejemplo, si se quiere mostrar tres tarjetas, cada una se define con clases como **col-sm-12 col-md-6 col-lg-4 col-xl-4**.
 - **En celular (pantallas extra pequeñas a pequeñas):** La clase **col-sm-12** hará que cada tarjeta ocupe las 12 partes de la rejilla, resultando en **una columna por fila**.
 - **En tablet (pantallas medianas):** La clase **col-md-6** hará que cada tarjeta ocupe 6 partes. Como $6+6=12$, se mostrarán **dos columnas por fila**, y la tercera tarjeta se moverá a la siguiente línea.
 - **En escritorio (pantallas grandes y extra grandes):** Las clases **col-lg-4** y **col-xl-4** harán que cada tarjeta ocupe 4 partes. Como $4+4+4=12$, se mostrarán **tres columnas por fila**.
 - Este comportamiento se puede observar al reducir el tamaño de la ventana del navegador, viendo cómo las tarjetas se reorganizan de 3 a 2 y luego a 1 columna.

5.5 Clases Utilitarias de Bootstrap

Las clases utilitarias de Bootstrap son pequeñas clases CSS predefinidas que te permiten aplicar estilos específicos rápidamente, como espaciado, alineación, sombras y colores, sin tener que escribir CSS personalizado para cada elemento.

- **Clases para espaciado (margins y paddings): m- (margin) y p- (padding), con opciones para top, bottom, izquierda, derecha, eje x y eje y (ej. my-3, mb-2).**
 - Bootstrap ofrece un sistema intuitivo de clases para controlar los márgenes (m-) y los rellenos (p-) de los elementos.
 - Estas clases se componen de:
 - m para *margin* o p para *padding*.
 - Una dirección:
 - t para *top* (arriba).
 - b para *bottom* (abajo).
 - s para *start* (izquierda, en idiomas LTR como español).
 - e para *end* (derecha, en idiomas LTR como español).
 - x para el eje horizontal (izquierda y derecha).
 - y para el eje vertical (arriba y abajo).
 - Un número del 0 al 5 que representa un tamaño predefinido en la escala de espaciado de Bootstrap.
 - **Ejemplos:**
 - mb-3: Aplica un *margin-bottom* (margen inferior) de 3 unidades a un elemento, separándolo del contenido de abajo.
 - my-3: Aplica un *margin-top* y un *margin-bottom* (margen en el eje Y) de 3 unidades, separando el elemento tanto arriba como abajo.
 - py-3: Aplica un *padding-top* y un *padding-bottom* (relleno en el eje Y) de 3 unidades.
- **Clases para alineación de texto: text-center, text-justify.**
 - Bootstrap proporciona clases para alinear texto de forma sencilla. La clase **text-center** centra el texto horizontalmente dentro de su contenedor.
- En un ejemplo, se utilizó una clase **text-justify** para justificar el texto de un párrafo, aunque en ese caso se implementó como una clase CSS personalizada, lo que demuestra la posibilidad de añadir estilos propios si Bootstrap no ofrece una clase específica para el comportamiento deseado.

- **Clases de *display*: `display-3` para darle un tamaño y estilo distinto a un título.**

Las clases de *display* permiten controlar la apariencia de los títulos y otros elementos. Por ejemplo, la clase **`display-3`** se utiliza para aplicar un tamaño y un estilo de fuente distintivo a un título `h1`, haciéndolo más grande y con una tipografía diferente a la predeterminada.

- **Clases para sombras: `.shadow`.**

La clase utilitaria **`.shadow`** se utiliza para añadir una pequeña sombra alrededor de un elemento, como una tarjeta, dándole un efecto visual de profundidad.

- **Clases para colores de fondo y texto: `bg-dark`, `navbar-dark`.**

- Bootstrap ofrece clases para aplicar rápidamente colores de fondo y de texto a los elementos. Por ejemplo, **`bg-dark`** se utiliza para dar un color de fondo oscuro (negro) a un elemento.
- La clase **`navbar-dark`** se aplica a una barra de navegación (`navbar`) para asegurar que los elementos de texto y los iconos dentro de ella (como los enlaces) se muestren con un color claro (generalmente blanco o gris claro) que contraste con el fondo oscuro, mejorando la legibilidad.

- **Clase para alineación vertical de ítems: `align-items-center` aplicada a un `.row` para centrar contenido verticalmente.**

Para alinear verticalmente el contenido dentro de una fila (`.row`), se puede usar la clase **`align-items-center`**. Esta clase se aplica al contenedor de la fila (el `div` con la clase `row`) y hace que los elementos hijos se centren verticalmente dentro de esa fila.

Existen otras variaciones, como `align-items-start` para alinear al inicio (arriba) y `align-items-end` para alinear al final (abajo). Esto es especialmente útil para mejorar la estética y la distribución del espacio en diseños responsivos, evitando espacios en blanco poco estéticos, por ejemplo, en una tablet.

5.6 Componentes y otras clases populares

Además de las clases de rejilla y utilitarias, Bootstrap ofrece una amplia gama de componentes predefinidos que son muy utilizados:

- **Botones (`.btn`, `.btn-primary`, `.btn-success`, etc.):** Permiten crear botones con diversos estilos y colores de forma rápida.
- **Tarjetas (`.card`):** Ideales para mostrar contenido de manera organizada, como noticias o productos, a menudo combinadas con clases de rejilla. Incluyen clases para imagen (`.card-img-top`), cuerpo (`.card-body`), título (`.card-title`), texto (`.card-text`) y pie de tarjeta (`.card-footer`).
- **Barras de Navegación (`.navbar`):** Proporcionan un componente responsivo para la navegación del sitio, con clases para el logo (`.navbar-brand`), enlaces (`.nav-link`) y un *toggler* para dispositivos móviles.
- **Formularios (`.form-control`, `.form-label`):** Estilizan elementos de formulario como inputs y textareas para una apariencia consistente y mejor usabilidad.

Estas clases y componentes, combinados con las capacidades de diseño responsivo de Bootstrap, hacen que sea una herramienta extremadamente eficiente para el desarrollo web en la actualidad y hacia el 2025.

6. Mejores Prácticas y Fundamentos de Accesibilidad

Esta sección se centra en identificar y evitar errores frecuentes en el desarrollo web, así como en comprender la importancia de hacer que los sitios sean accesibles para todos los usuarios.

6.1 Errores Comunes en el Desarrollo Web (a evitar)

El desarrollo web a menudo presenta trampas comunes que pueden afectar la calidad, el rendimiento y el posicionamiento de una página. Evitarlas es fundamental para un proyecto exitoso.

- **No utilizar correctamente el código HTML: Usar etiquetas para lo que fueron diseñadas (semántica).**
 - Es un error común encontrar sitios web que no utilizan adecuadamente el lenguaje HTML, a pesar de que este permite muchas alternativas para lograr un resultado visual similar. Por ejemplo, usar `<p><font size=24>Nueva Thermomix</font></p>` para un título es "terrible", aunque visualmente sea similar a `<h1>Nueva Thermomix</h1>`, porque el HTML debe limitarse a definir la estructura del contenido sin detallar su aspecto visual.
 - El **HTML semántico** es aquel que introduce significado a la página, no solo presentación. Las etiquetas deben usarse para el propósito para el que están diseñadas, de modo que al verlas, se sepa qué parte es importante, cuál no tanto, o qué es un menú (`<nav>`). Un buen código HTML, que aprovecha todas las etiquetas semánticas, es crucial para el posicionamiento de la página en motores de búsqueda. Etiquetas como `<b>` (negrita) o `<i>` (cursiva) no son semánticas, ya que definen cómo se muestra el texto sin aportar significado. Por otro lado, `<div>` no es semántica porque podría contener cualquier cosa, pero `<p>` (párrafo) o `<h1>` (título principal) sí lo son porque le dan significado al contenido. Es importante dar preferencia al HTML semántico frente al no semántico, especialmente si el CSS no está disponible o no es legible por un dispositivo de asistencia, para que el usuario pueda seguir comprendiendo el significado del contenido.
 - Una estructura HTML5 semántica actualizada con etiquetas como `<article>`, `<header>`, `<nav>`, `<main>` y `<footer>` no solo es buena para el SEO, sino que también facilita la navegación para usuarios con tecnologías de asistencia, como los lectores de pantalla.
- **Incrustar CSS o JavaScript dentro del HTML: Mantener el código limpio y organizado en ficheros separados.**
 - Es un error típico **mezclar código CSS con el código HTML**. Aunque está permitido y puede ser útil en etapas de desarrollo, una web en producción debe tener un código limpio y organizado, con ficheros HTML y CSS **totalmente separados**.
 - De manera similar, el código JavaScript tampoco debe mezclarse entre las etiquetas HTML. Es preferible que el JavaScript se incluya al final del documento HTML para que el renderizado de la web pueda comenzar antes, mejorando la experiencia del usuario. Los archivos de estilos y scripts deben ser externos y enlazados correctamente en el `<head>` para CSS y al final del `<body>` para JavaScript.
- **Utilizar fotografías pesadas: Optimizar imágenes para no ralentizar la carga de la página.**
 - Las cámaras de fotografía generan archivos muy grandes, no adecuados para páginas web, ya que estorban y **ralentizan la carga de la página**. Es fundamental **optimizar las imágenes**.
 - Las imágenes son, a menudo, los elementos más pesados de una página web. Es crucial usar formatos de imagen de próxima generación como **WebP**, que ofrecen buena calidad con menor tamaño de archivo. También se recomienda implementar la **carga diferida (lazy loading)** para las imágenes que no están en la vista inicial, usando el atributo `loading="lazy"` en HTML5. Además, la compresión y el dimensionamiento correcto de las imágenes son vitales para una carga rápida sin sacrificar la calidad visual.
- **Utilizar fuentes TTF de tu PC o Flash: Usar webfonts y evitar tecnologías obsoletas.**
 - Otro error típico es usar fuentes TTF instaladas solo en el ordenador del desarrollador, ya que la mayoría de los usuarios no las tendrán disponibles. La alternativa recomendada son las **webfonts**.

- Se debe **evitar el uso de tecnologías completamente obsoletas como Flash**. No hay buenas razones para utilizarlo, ya que requiere plugins adicionales que suelen causar bloqueos en los dispositivos.
- **Utilizar editores tipo Word para HTML: Generan código “terrible”; es mejor usar editores de texto plano o WYSIWYG de forma correcta.**
 - Utilizar editores tipo Word para crear HTML es un error grave. Word es un excelente procesador de textos, pero **no sirve para hacer webs**, ni siquiera partes de ellas. Copiar y pegar de Word a un editor WYSIWYG (What You See Is What You Get) como Frontpage o Dreamweaver, o directamente a un gestor de contenidos (WordPress, Joomla), genera código "terrible".
 - El lenguaje HTML es muy sencillo de aprender. Para introducir contenido, es mejor hacerlo correctamente desde cero o **copiar y pegar desde un editor de texto plano** como Bloc de Notas, UltraEdit, Notepad++, o Sublime Text. Si se usa un editor WYSIWYG, se debe redactar los textos allí mismo o pegar desde texto plano y usar la barra de botones que proporciona la herramienta para aplicar el formato deseado. Herramientas como Visual Studio Code y Sublime Text son editores de código potentes y personalizables que facilitan la escritura de HTML y CSS.

6.2 Fundamentos de Accesibilidad Web (A11Y)

La accesibilidad web es un pilar fundamental del desarrollo frontend, asegurando que el contenido digital sea utilizable por todos.

- **Definición: Hacer que los sitios web puedan ser utilizados por el mayor número de personas posible, incluyendo aquellos con discapacidades.**

La **accesibilidad web** significa que las personas con algún tipo de discapacidad podrán hacer uso de la Web gracias a un diseño que les permita percibir, entender, navegar e interactuar con ella. Consiste en hacer que los sitios web, plataformas y tecnologías de Internet sean diseñadas y desarrolladas para que cualquier persona pueda consultarlos, adquirirlos, percibirlos y operarlos, independientemente de su discapacidad, barreras de lenguaje, recursos tecnológicos o condiciones socioeconómicas. Su objetivo principal es permitir a cualquier usuario, sin importar su discapacidad, el acceso a los contenidos y la navegación necesaria en el sitio.

- **Beneficios: Es una práctica ética, aumenta el alcance del sitio, mejora el SEO y puede ser un requisito legal. Google premia los sitios accesibles con un mejor ranking.**
 - La accesibilidad no es solo una opción ética, sino un elemento crítico del desarrollo frontend.
 - Garantizar la accesibilidad **aumenta el alcance del sitio**, ya que una gran cantidad de usuarios tienen algún tipo de discapacidad. Si una página no es accesible, se pierden muchos usuarios o estos sufren para interactuar con el contenido.
 - La accesibilidad **mejora el SEO y la visibilidad de la marca**. Google favorece el posicionamiento de los sitios con diseño responsivo y accesibles, premiándolos con un mejor ranking. Un *frontend* bien estructurado con HTML semántico y prácticas SEO mejora el posicionamiento orgánico.
 - Además, la implementación de la accesibilidad es un **requisito legal** en muchos lugares, como en Europa (estándar EN 301 549) y EE.UU. (Section 508). Permite a las personas ejercer sus derechos, como acceder a la educación, encontrar empleo o vivir de manera independiente. También contribuye a reducir el costo de desarrollo y mantenimiento, a la reutilización de contenidos, y a una mejor gestión de la carga del servidor y ancho de banda.
- **Principios WCAG (Web Content Accessibility Guidelines): La accesibilidad se basa en cuatro principios fundamentales: Perceptible, Operable, Comprensible y Robusto.**
 - Las **Pautas de Accesibilidad para el Contenido Web (WCAG)** son estándares técnicos que establecen características para la presentación y acceso a los elementos de un sitio web. Estos principios, en su versión 2.1 y 2.2, son la base de la accesibilidad.
 - **Perceptible:** La información y los componentes de la interfaz de usuario deben presentarse de forma que los usuarios puedan percibirlos con facilidad a través de los sentidos, usando opciones alternativas de comunicación. Lo que no se oye, se debe poder ver; lo que no se ve, se debe poder escuchar o tocar.

- **Operable:** Los componentes de la interfaz de usuario y la navegación deben ser manejables. Todas las funciones deben poder ser realizadas por cualquier persona, por ejemplo, sin necesidad de un ratón, utilizando el teclado u otros métodos de entrada.
- **Comprensible:** La información y las operaciones de los usuarios deben ser comprensibles. El contenido debe ser claro y la interacción intuitiva, para que no sea difícil entender cómo realizar una acción.
- **Robusto:** El contenido debe ser suficientemente robusto para ser interpretado por una gran variedad de agentes de usuario, incluyendo tecnologías de asistencia. Esto implica que el sitio debe funcionar sin restricciones por el tipo de navegador o *hardware*, y ser capaz de manejar errores.

➤ **Técnicas básicas:**

- **Uso de texto alternativo (`alt`) en imágenes.**

Es una técnica fundamental para la accesibilidad. La propiedad `alt` en las imágenes debe ser utilizada para que los lectores de pantalla puedan describir su contenido a usuarios con discapacidades visuales. Este texto alternativo debe ser descriptivo y no repetitivo, siempre que la información no esté presente en otra parte.

- **Estructura HTML semántica.**

La implementación de una estructura HTML semántica es crítica para la accesibilidad. Las etiquetas semánticas ayudan a los motores de búsqueda y a los dispositivos de asistencia a interpretar mejor el contenido, como el `<nav>` para la navegación principal o `<header>` para la cabecera. Es importante que el contenido se estructure jerárquicamente con encabezados (`<h1>`, `<h2>`, etc.) para una navegación lógica.

- **Asegurar la navegación con teclado.**

Los controles de la página deben ser accesibles por el teclado u otro tipo de dispositivo de asistencia, sin necesidad de usar el ratón. Esto incluye validar que toda la funcionalidad pueda ser operada con un teclado, sin "trampas de teclado" que impidan salir de un componente. El foco del teclado debe ser visible y seguir un orden lógico.

- **Usar contrastes y colores adecuados para la legibilidad.**

Asegurarse de que el texto sea legible mediante un **contraste suficiente entre el texto y el fondo** es crucial para personas con dificultades visuales. Bootstrap mismo utiliza clases de color para asegurar la legibilidad, como `navbar-dark` que hace que los textos e iconos sean claros sobre un fondo oscuro. Las WCAG establecen una relación de contraste mínima de 4.5:1 para la presentación visual de imágenes y texto. También es importante que los elementos no dependan únicamente del color, forma, tamaño o ubicación para ser identificados. Herramientas como Color Contrast Analyzer y WAVE pueden ayudar a verificar los niveles de contraste y la legibilidad.